

Deep Dive: Inheritance, Object Properties, and the `super()` Function

CMPT 211: Introduction to Software Development

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Recap: The Base Blueprint and Object Properties

Last week, we designed a foundational blueprint for all students at the university.

```
class ConcordiaStudent:
    # The Constructor Method
    def __init__(self, name, student_id):
        # Setting the foundational attributes (properties)
        self.name = name
        self.student_id = student_id
        self.university = "Concordia University of Edmonton"
        self._gpa = 0.0

    def get_email(self):
        return f"{self.name.lower()}@student.concordia.ab.ca"
```

What is an Object Property? Properties (also called attributes) are variables bound directly to an object using the `self.` prefix. They represent the permanent "state" or "memory" of that specific object.

The Limitation of the Base Class

Our `ConcordiaStudent` class works perfectly for tracking general university data. However, a university is a complex ecosystem.

The Scenario:

- The Information Technology (IT) department needs to track **Server Access Levels** for their students.
- The Computer Science (CS) department needs to track **GitHub Usernames** for their students.

If we add `server_access` and `github` to the base `ConcordiaStudent` class, we waste memory. A biology major does not need a GitHub property, and an English major does not need server access. The base class must remain generic!

The Problem: Don't Repeat Yourself (DRY)

We *could* copy and paste the entire `ConcordiaStudent` code into a new file, rename it `ITStudent`, and add the new properties manually.

Why is copy-pasting terrible for software engineering?

- **Maintenance Nightmare:** Imagine the university changes its email domain to `@cue.ca`. If you copy-pasted the class 50 times for 50 different majors, you have to find and manually update the `get_email()` method in 50 different places. If you miss one, you introduce a catastrophic bug.
- **Memory Bloat:** We are duplicating identical code logic unnecessarily.
- **Violates the DRY Principle:** Professional developers strive to write a piece of logic exactly *once* and reuse it everywhere.

The Solution: Class Inheritance

Inheritance is a mechanism that establishes an "IS-A" relationship. An `ITStudent` IS A `ConcordiaStudent`.

To tell Python to link the two blueprints, we pass the Parent's name into the Child's class definition inside parentheses.

```
# The Parent Class (Superclass)
class ConcordiaStudent:
    pass # Foundational logic lives here

# The Child Class (Subclass)
# The parentheses explicitly link ITStudent to ConcordiaStudent
class ITStudent(ConcordiaStudent):
    pass
```

Because of those parentheses, `ITStudent` instantly absorbs every single property and method from the parent class automatically, as if you had typed them out yourself!

The Challenge: Initializing Subclasses

Inheritance gives us the methods automatically, but object **properties** are generated dynamically when the object is instantiated (during `__init__`).

When we instantiate an `ITStudent`, we need it to have the `name` and `student_id` properties from the parent, PLUS its new `server_access` property.

```
class ITStudent(ConcordiaStudent):
    # 1. The child's constructor must ask for ALL required data upfront
    def __init__(self, name, student_id, server_access):

        # 2. We don't want to manually set self.name = name here!
        # That violates DRY. We want the Parent class to do it.

        # ??? HOW DO WE SEND DATA UP TO THE PARENT? ???

        # 3. Setting the new child-specific property
        self.server_access = server_access
```

The super() Function Explained

We use the `super()` function. It is a built-in Python command that returns a temporary object of the superclass (the parent). It acts as a direct hotline, allowing the child to call the parent's methods.

```
class ITStudent(ConcordiaStudent):  
  
    def __init__(self, name, student_id, server_access):  
        # 1. Call the Parent's constructor via super()  
        # We pass 'name' and 'student_id' UP the chain.  
        # CRITICAL RULE: We do NOT pass 'self' inside super().__init__()  
        super().__init__(name, student_id)  
  
        # 2. Handle the specific Child properties  
        self.major = "Information Technology"  
        self.server_access = server_access
```

Tracing the Execution of `super()`

What exactly happens in memory when `super().__init__` is called?

- 1 **Pause:** Python pauses the execution of the `ITStudent` constructor.
- 2 **Travel Up:** Python jumps up to the `ConcordiaStudent` class definition.
- 3 **Execute Parent Setup:** It executes the parent's `__init__`, binding `self.name`, `self.student_id`, and `self.university` to our new object.
- 4 **Return:** Python jumps back down to the `ITStudent` constructor right where it left off.
- 5 **Execute Child Setup:** It finishes by adding `self.major` and `self.server_access` to the exact same object.

Result: One unified object containing properties built by two different classes!

Branching Out: The CSStudent

We can create multiple specific subclasses from the same parent. They become "Siblings". They share the parent's base properties, but have completely distinct specialized properties and methods.

```
class CSStudent(ConcordiaStudent):  
  
    def __init__(self, name, student_id, github_user):  
        # 1. Pass the common data up to the Parent  
        super().__init__(name, student_id)  
  
        # 2. Add CS-specific properties  
        self.major = "Computer Science"  
        self.github = github_user  
  
    # 3. Method Extension: Adding new behaviors  
    def push_code(self):  
        print(f"{self.name} is committing to {self.github}.")
```

Execution: Proving the Inheritance and Properties

Let's put the objects into memory and test how Python resolves these properties and methods at runtime.

```
# Instantiation
alice = ITStudent("Alice", 11111, "Admin")
bob = CSSStudent("Bob", 22222, "bob_dev")

# 1. Testing Parent Methods
# Both inherited the get_email() method without us rewriting it!
print(alice.get_email()) # Output: alice@student.concordia.ab.ca
print(bob.get_email())   # Output: bob@student.concordia.ab.ca

# 2. Testing Subclass Properties
print(alice.server_access) # Output: Admin
print(bob.github)          # Output: bob_dev

# alice.push_code() -> CRASH! AttributeError.
# Siblings do not share their unique methods!
```

Lecture Summary

- **Object Properties:** Variables bound to an instance using `self`. They represent the state of the object.
- **Inheritance:** We create specialized Subclasses from generic Superclasses to maximize code reuse and prevent maintenance nightmares (DRY principle).
- **Syntax:** Declared by placing the parent's name in parentheses: `class SubClass(SuperClass):`
- **The `super()` Function:** Used to delegate setup tasks up the inheritance chain. By calling `super().__init__()`, we guarantee the foundational properties are properly built by the parent before we add our specialized modifications.
- **Method Extension:** Subclasses inherit all parent behaviors, but can also define brand new methods unique to themselves.

Any Questions?

Review these class structures on Moodle before the lab.